Preliminary Concepts

1

1.1 Introduction to Data Structures

Having learned how to store data in a computer's memory from a previous text or course using variables, arrays, strings, and objects, it should be seen that there are a vast number of possibilities to store and manipulate data. However, instead of having to "reinvent the wheel" each time a program is written, others have found that there are some common techniques to store data that are quite helpful in a wide variety of situations. Some of these include stacks, queues, lists, and trees, all of which will be discussed in this text. The different ways data can be stored, along with the specific operations that can be performed on data, is known as *data structures*.

Although individuals might sometimes refer to an array as a data structure, this is not very precise. It is not the mechanism of storing the data that is a data structure, but rather it is the combination of the way the data is stored and the associated methods that can be performed on the data that define the data structure. For example, an array itself is not a data structure, but rather the order of the data (for example, integers in ascending order) and the associated operations that can be performed on the data which might be implemented using an array that constitutes a data structure.

1.2 Prior Concepts and Skills Needed

The prior concepts and skills needed for this text should have been learned in a previous introductory class or text using the Java programming language. In particular, the reader should have a good grasp of input/output, assignment statements, arithmetic and Boolean expressions, selection structures (`if` and `switch` statements), and iteration structures (`for`, `while`, and `do-while` statements). Furthermore, the reader should have a working knowledge of primitive data types,

J. T. Streib and T. Soma, *Guide to Data Structures*, Undergraduate Topics in Computer Science, https://doi.org/10.1007/978-3-319-70085-4_1

strings, and one and two-dimensional arrays. Lastly, a good background in object-oriented programing is needed including data members, value-returning and void methods, constructors, private, protected, and public data members and methods, sending and receiving objects from a method, instance and class constants, variables and methods, inheritance, abstract classes, and polymorphism.

The authors understand that learning data structures can be a bit intimidating at times, so in many cases the authors will remind the reader of previous subtle points and concepts to help in understanding the topic under consideration. However, if at any time the reader feels deficient in any of the areas mentioned above, it is recommended that time be taken to review these concepts in an introductory text such as *Guide to Java* [2] or other similar text. Although some readers might have had some exposure to elementary algorithm analysis, abstraction, and interfaces, these topics are reviewed here in this preliminary chapter to ensure that the reader is ready for the first chapter on stacks.

1.3 Elementary Algorithm Analysis

Just as there are different ways of storing data, there are different ways of processing data. If an algorithm is running slow, what can be done to speed it up? A first response might be to get a faster processor and if a processor is twice as fast, then one might expect the algorithm to be approximately twice as fast too. But what if the goal was to have an algorithm run 1000 times faster? It might be difficult and expensive to find a processor that runs that fast. If possible, an alternative is to write a faster algorithm. Is there a way to determine whether one algorithm is faster than another algorithm? The answer is yes, and it is through the process of algorithm analysis. For example, if a program had two statements as follows, how fast is the algorithm?

```
total = total + x;
i = i + 1;
```

Although some operations might take longer than others and speed might vary from computer to computer, assume that each operation (such as assignment, comparison, and arithmetic) will count as one unit of time. In the current example, each arithmetic statement would take 2 units (one for the arithmetic and one for the assignment) for a total of 4 for the two statements. Continuing, what if there were three statements as shown below?

```
x = y * z;
total = total + x;
i = i + 1;
```

The additional statement adds 2 time units, which increases the total to 6. Although the first code segment is approximately 33% faster, is this a significant amount of time? In one sense it is, but in comparison to other algorithms it is not a significant increase. As will be seen shortly, regardless of how much data is processed, each code segment is executed in a fixed amount of time and it can be said that both of these code segments are executed in constant time.

A common notation, called *Big O* notation (pronounced Big Oh), uses the capital letter O to compare the relative order of magnitude of various algorithms. Algorithms that are executed in constant time are said to be of order 1 or O(1). The number 1 is used regardless of how many actual time units the algorithm took and to indicate that the time is constant. So in answering the question whether one of the above code segments is significantly faster than the other, neither is because they are both O(1) algorithms.

However, what if the above three lines of code were placed in a `while` loop along with another line to initialize the variable `i` as shown below?

```
i = 1;
while(i <= n) {
  x = y * z;
  total = total + x;
  i = i + 1;
}
```

In the above example, the first assignment statement will take one time unit. As before, the three arithmetic statements in the body of the `while` loop would take 6 time units. If the value of `n` was 3, then each statement in the body of the loop would be executed three times, for a total of 18 time units. Further, how many times does the `while` statement itself get executed? If one answered 3, that answer would unfortunately be incorrect. Don't forget that the final value of `i` is checked one last time before control is transferred to the bottom of the loop. The correct answer is 4 times. Again, assuming the value of `n` is 3, the result is that it would take 23 time units to execute the code segment as shown below:

```
i = 1;                      // 1 operation
while(i <= n) {             // 3 times thru loop + 1 last check of i
    x = y * z;              // 2 operations * 3
    total = total + x;      // 2 operations * 3
    i = i + 1;              // 2 operations * 3
}
```

An astute programmer might recognize that the first assignment statement in the body of the loop does not need to be calculated each time through the loop. As a result, that statement could be moved prior to the execution of the loop as follows:

```
x = y * z;
i = 1;
while(i <= n) {
  total = total + x;
  i = i + 1;
}
```

Given this change, how many units of time would it take to execute this code segment assuming again that the value of `n` is 3? Using the same method of analysis discussed above, the correct answer is 19. Another way to look at it is that since the `x = y * z` is executed only once instead of three times, there are 4 fewer operations that need to be performed.

What if the value of `n` in the above two program segments were 10? The answers would be 72 and 54, respectively. Again, the `x = y * z` would be executed nine fewer times which accounts for the speed up of 18 units in the second segment. What about 100? The answers would be 702 and 504, respectively. In the first segment, notice that there are 7 operations for each iteration of the loop (the comparison, the multiplication, the two additions, and the three assignments), where 7 times 100 is 700. Plus there is the initialization of `i` and the last check of `i < n` prior to exiting the loop, for a total of 702. Likewise with the second example, there are 5 operations for each iteration of the loop (the two additions, two assignments, and one comparison), plus the 4 extra operations outside the loop (the two assignments, the multiplication and the last comparison before exiting the loop) for a total of 504.

The result is that by careful programming, there can be some change in algorithmic efficiency. In this case there is a 28.2% speed up. Is this a very large change? Further, could the answers above be more generic in terms of n? The answer to the second question is yes, where one might see the pattern in the numbers above, especially when n equals 100. The first algorithm would be 7n + 2 and the second algorithmwould be 5n + 4. In looking at these two expressions, does the +2 or +4 make any significant changes to its efficiency? It can be seen as the value of n gets higher, say one million, the effects of these values are rather insignificant.

Further, whether the result is 5n or 7n, they are both linear. Regardless if the coefficient is a 5 or a 7, the algorithm depends on the value of n. The result is that both of these algorithms are considered to be order n or O(n). So in answer to the first question in the preceding paragraph, in both cases these algorithms are considered to be the same order of magnitude and it is not a significant change.

Does this mean that one does not need to do a detailed analysis of an algorithm? Although these two algorithms can be determined by inspection to be of order n because they both loop n times, there are some other very tricky algorithms where it might be necessary to take the time to do a line-by-line analysis. Further, does this mean that one should not try to increase the efficiency of an algorithm from 7n to

5n? As will be seen there are other algorithms that are faster than O(n), but if a faster algorithm is not available and speed is a concern, then one should try to increase an algorithm's efficiency, especially inside a loop or nested loops. However, as will be seen later in this text, sometimes there can be a dramatic increase in the speed of an algorithm by using an alternative method.

In looking at some other algorithms, what would be the efficiency of the following code segment?

```
for(i=1;i<=n;i++) {
  // some statements
}
for(i=1;i<=n;i++) {
  // some statements
}
```

Since the specific statements in the body of the loops are not included, it is not possible to do a detailed analysis. However, by inspection it should be clear that the first loop would be O(n) and the second loop would also be O(n). But what is the efficiency when they are combined? Since they are both of O(n) and one follows the other, the two are added and the result would appear to be n + n or of order 2n. Further, as discussed earlier the coefficient can be dropped and the algorithm consisting of the two non-nested loops is O(n). In another example, consider the following algorithm that also consists of two `for` loops. What is its algorithmic efficiency?

```
for(i=1;i<=n;i++)
  for(j=1;j<=n;j++) {
    // some statements
  }
```

Unlike the previous code segment, the loops in this segment are nested. If the answer is not readily apparent, it is sometimes helpful to pick a value for `n` to help determine how many times the statements in the inner loop are executed. The key is not to pick a number too small where the pattern might not be evident, nor pick a number too large in case one has to manually walk through the code. If the value for `n` was 3, how many times would the statements in the inner loop be executed? The outer loop would execute three times, and for each time through the outer loop, the inner loop would execute three times too. The result is that the statements in the inner loop would be executed nine times. What if the value of `n` was 10? Then the statements in the inner loop would be executed 100 times. The result is that for any value `n`, the inner statements would be executed $n \times n$ or n^2 times and thus the segment is $O(n^2)$. Given the previous two example, consider the following:

```
for(i=1;i<=n;i++)
  for(j=1;j<=n;j++) {
    // some statements
  }
for(i=1;i<=n;i++) {
   // some statements
}
```

At first it might appear that the answer might be $O(n^3)$ because there are three loops. However, notice that the first two loops are nested and the last one is not nested. Since the first two are nested they are n^2 and since the last loop is not nested it is n. The result is that the two are added to form $n^2 + n$. But what is this in Big O notation? When there are more than one term in the resulting analysis, only the most dominate term is used in Big O notation, or in other words the slowest term is used. The result is that the above algorithm is $O(n^2)$. In another example, if an algorithm was analyzed to be $3 + \log n + 2n$, what would it be in Big O notation? The answer would be O(n) because 2n is the dominate term (slowest) and as before the coefficient is not included.

Continuing, what would the efficiency be for the following loop?

```
i=n;
while(i>1){
  // some statements
  i = i / 2;
}
```

At first it might appear that the inner statements are executed ½ n times due to the division by 2, but this would be a mistake. As mentioned above, it is sometimes helpful to pick a number for `n` and walk through the code segment. If one chose 4, then one might have also come to the conclusion that the answer is ½ n since it would loop 2 times. However, as mentioned previously, one does not want to pick a number that is too small where a pattern might not be seen. Instead, try the number 16 for the value of `n`. The first time through the value of `i` would be 16. The second time through the value of `i` would be 8. The third time through it would be 4, followed by the value of 2 the fourth time through the loop. The loop does not iterate a fifth time because the value of `i` would be 1 which makes the relation in the `while` statement `false`. The result is that when `n` is 16, the loop iterates 4 times. How many times would it iterate when `n` was 32? As one might suspect, the answer is 5. If 2^4 is equal to 16, then $\log_2$ of 16 is 4. Likewise, if 2^5 is equal to 32, then $\log_2$ 32 is equal to 5. The result is that the above code segment is logarithmic and is of order $\log_2$ n or O(log n). Notice that if the subscript 2 is left off the logarithm, because in the field of computer science, log is assumed to be $\log_2$.

In one more example, what is the algorithmic efficiency of the following code segment?

```
for(i=1;i<=n;i++) {
  j=n;
  while(j>1) {
    // some statements
    j = j / 2;
  }
}
```

As with the nested `for` loops previously, where the two nested O(n) loops were n × n or $O(n^2)$, similarly the outer loop is O(n) and the inner loop is O(log n) which combine to give n × log n or O(n log n). As will be seen in Chap. 9, some of the fastest sorting algorithms are O(n log n).

In all the previous examples, regardless of the coefficients in a particular instance, as n gets sufficiently large, an O(log n) algorithm is faster than an O(n) algorithm, an O(n) is faster than an O(n log n) algorithm, and an O(n log n) algorithm is faster than an $O(n^2)$ algorithm. Of course the fastest is an O(1) algorithm because it does not depend on the value of n as shown in Fig. 1.1.

This concept of comparing algorithms is very important in the field of computer science, since the relative speed of algorithms can be compared with each other. Although briefly introduced here, this concept will be seen again throughout the text.

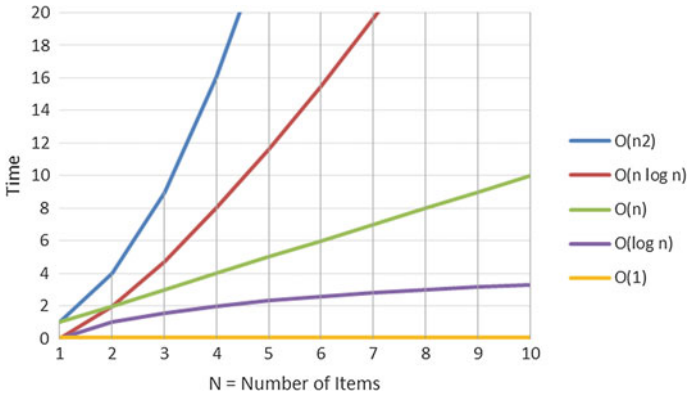

Fig. 1.1 Comparison of $O(n^2)$, O(n log n), O(n), O(log n), and O(1)

1.4 Abstraction and Interfaces

In addition to the ability to create various data structures using an array, many can also be implemented with objects that can be linked together using references, which will also be discussed in this text. How these data structures are implemented is often hidden from the person using the data structure and this concept is known as *data abstraction*. For instance, whether a list is implemented using an array or using references is an example of data abstraction. Further, how the details of an algorithm are implemented is known as *procedural abstraction*. As might have been learned in a previous course or text, how a number is calculated using either iteration or recursion is an example of procedural abstraction. Together, through the use of data abstraction and procedural abstraction, one can create a data structure known as an *abstract data type* (ADT).

As an illustration, whether a vehicle has a four-cylinder engine or a rotary engine is an example of data abstraction and whether the vehicle uses a mechanical or electrical steering process can be an example of procedural abstraction. Although there are advantages and disadvantages to various types of engines (for example, a diesel engine has a lot of torque and is often used in trucks), the driver of a vehicle might not know what type of engine is being used and all that he or she cares about is that when the accelerator is depressed the vehicle accelerates. Likewise, the driver of a vehicle might not be concerned whether the steering is implemented through a mechanical or electrical process, but rather only cares that when the steering wheel is turned to the left, the vehicle turns to the left. The combination of various mechanical parts and processes in the vehicle make it an abstract data type.

Just as there are advantages and disadvantages to using different mechanical parts in a vehicle, there are advantages and disadvantages to how data is stored. Some ways of storing data are fixed and they cannot change or it is difficult to change their size during the execution of a program and are known as *static*, such as arrays. Others have the ability to change during the execution of a program, such as using references, and these are known as *dynamic*. Further, dynamic structures can be faster than static ones. At first, it might seem as though the dynamic structures would be the best choice under all circumstances. However, it should also be pointed out that dynamic structures generally take up more memory than static structures. The result is that determining which type of structure should be chosen depends upon the circumstances.

Consider a simple application which calculates and outputs the average of three exams. How could the three scores be stored? One way is to create three separate memory locations and another is to use a one-dimensional array of size 3. How the scores are stored, either using three separate variables or a one-dimensional array, could be hidden from the application programmers. They could use the class without needing to know the details of how it was coded. To facilitate this, a Java *interface* structure can be used to separate the headings of the methods from the implementation of the methods. A Java interface lists a set of abstract methods without their code. A class that implements an interface provides definitions for the

methods. The important point is that the particular implementation which is used will not affect other programs that interact with it because every implementation is written to satisfy method headings in the interface. The reason for having several different implementations for the same set of methods is that one class may be more efficient than another at performing certain kinds of operations with certain type of data. Programmers can choose the best implementation depending on the nature of the application.

The following code is a Java interface which deals with exam scores. The definition starts with the visibility modifier `public` followed by the keyword `interface`, which is followed by the name of the interface `ScoresAvgADT`.

```
public interface ScoresAvgADT {
  // asks user to enter test scores
  public void enterScores();

  // computes and outputs average of test scores
  public void computeAverage();
}
```

As can been seen, it has two method headings, `enterScores` which should ask a user to enter the scores and `computeAverage` that should compute and output the average of the test scores. These methods can be implemented with three variables or a one-dimensional array in separate classes. The classes that implement an interface must provide all of the methods listed in the interface, and those methods must be declared as `public`.

The keyword `implements` is used to declare a class that implements the interface. To illustrate, a complete class using three separate variables is shown below.

```
import java.util.*;
public class Scores implements ScoresAvgADT {
  private final static int NUM_EXAMS = 3;
  private Scanner scanner = new Scanner(System.in);
  private int exam1,exam2,exam3;

  public Scores() {
  }

  public void enterScores() {
    System.out.print("Enter three scores: ");
    exam1 = scanner.nextInt();
    exam2 = scanner.nextInt();
    exam3 = scanner.nextInt();
  }
```

```
  public void computeAverage() {
    double sum;
    sum = exam1 +  exam2 + exam3;
    System.out.printf("Average score: %.2f \n",
    sum / NUM_EXAMS);
  }
}
```

In contrast, a complete class which implements an interface using a one-dimensional array is shown below. It does the same tasks as the previous class, but the only difference is that an array and loop are used, illustrating the principle of data and procedural abstraction.

```
import java.util.*;
public class Scores1D implements ScoresAvgADT {
  private final static int NUM_EXAMS = 3;
  private Scanner scanner = new Scanner(System.in);
  private int[] exam;

  public Scores1D() {
    exam = new int[NUM_EXAMS];
  }

  public void enterScores() {
    System.out.print("Enter three scores: ");
    for(int i=0; i<NUM_EXAMS; i++)
      exam[i] = scanner.nextInt();
  }

  public void computeAverage() {
    double sum;
    sum = 0.0;
    for(int i=0; i<NUM_EXAMS; i++)
      sum = sum + exam[i];
    System.out.printf("Average score: %.2f \n",
    sum / NUM_EXAMS);
  }
}
```

Further, Fig. 1.2 is a *UML* (Unified Modeling Language) diagram that shows the `ScoresAvgADT` interface and two classes each of which contain two methods. Note that a dashed line from the class to the interface is used to indicate the relationship between the interface and the classes that implement it.

Once a class implements an interface, it can be used to create an instance of a class such as `Scores` using a variable of type `ScoreAvgADT`. For example, if the class `Scores` implements `ScoresAvgADT`, then the following declaration and creation of the object using the variable `scores` is correct:

```
ScoresAvgADT scores = new Scores();
```

In general, it is a good idea to declare variables for an object using interface types rather than class types. This allows flexibility because it does not tie the variables to particular implementations. If variables are declared with interface types, then changing to another implementation for that particular reference is usually just a matter of calling a different constructor, such as:

```
ScoresAvgADT scores = new Scores1D();
```

None of the other code in the calling program using the variable `scores` should have to be changed. A simple main program invoking instance of both classes is shown below:

```
public class TestScores {
  public static void main(String[] args) {
    ScoresAvgADT scores;

    scores = new Scores();
    scores.enterScores();
```

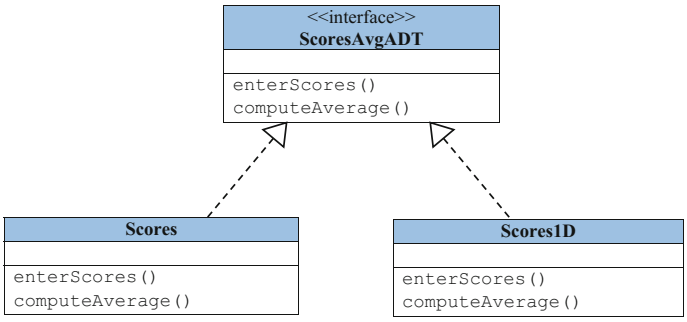

Fig. 1.2 UML diagram showing the `ScoresAvgADT` and its implementing classes

```
        scores.computeAverage();

        scores = new Scores1D();
        scores.enterScores();
        scores.computeAverage();
    }
}
```

Interfaces are used to express abstract data types in Java. They define a behavior of the object by listing headings of the methods with comments that specify how each method is to be used without showing the actual code. Thus, encapsulation is enforced by hiding the details of the implementation. These methods can be used only if there is a class which implements an interface that provides the code of all the methods in the interface. Whereas an interface includes only constants and abstract methods, the class implementing the interface can have other public and private methods and data members in addition to the implementation of all the methods from the interface.

1.5 Java API

Note that there exist in the Java programming language various libraries that contain a number of data structures such as stacks and queues. These libraries are known as the Java Application Program Interface (*API*) and they will be introduced in Chap. 11. The casual reader might think that it would be good enough to just learn how to use these libraries and be done with it. Under certain circumstances, using these libraries might be sufficient "to get the job done." However, there are certain deficiencies to these libraries and should one want to create their own data structures, then knowledge of how to do so is in order.

Further, being able to just use the libraries would be similar to a mechanical engineer only being able to drive an automobile and not being able to understand how it works or be able to design an automobile. Similarly, a computer scientist must not only know how to use existing data structures, but understand how they work, know their limitations, and be able to design and create his or her own data structures. This can be accomplished by learning how the data structures work, their advantages and disadvantages, and how to design, create, test, and debug them. To that end, Chap. 2 begins with using one of the simplest yet important data structures, the stack.

1.6 Complete Program: Implementing `Interface`

The complete program discussed in Sect. 1.4 is as shown below. Note that each of the classes and interface may need to be placed in separate modules in an *IDE* (Integrated Development Environment) in order to compile and work properly.

```
public class TestScores {
  public static void main(String[] args) {
    ScoresAvgADT scores;

    scores = new Scores();
    scores.enterScores();
    scores.computeAverage();

    scores = new Scores1D();
    scores.enterScores();
    scores.computeAverage();
  }
}
public interface ScoresAvgADT {
  // asks user to enter test scores
  public void enterScores();

  // computes and outputs average of test scores
  public void computeAverage();
}
import java.util.*;
public class Scores implements ScoresAvgADT {
  private final static int NUM_EXAMS = 3;
  private Scanner scanner = new Scanner(System.in);
  private int exam1,exam2,exam3;

  public Scores() {
  }

  public void enterScores() {
    System.out.print("Enter three scores: ");
    exam1 = scanner.nextInt();
    exam2 = scanner.nextInt();
    exam3 = scanner.nextInt();
  }

  public void computeAverage() {
    double sum;
```

```
    sum = exam1 + exam2 + exam3;
    System.out.printf("Average score: %.2f \n",
    sum / NUM_EXAMS);
  }
}
import java.util.*;
public class Scores1D implements ScoresAvgADT {
  private final static int NUM_EXAMS = 3;
  private Scanner scanner = new Scanner(System.in);
  private int[] exam;

  public Scores1D() {
    exam = new int[NUM_EXAMS];
  }

  public void enterScores() {
    System.out.print("Enter three scores: ");
    for(int i=0; i<NUM_EXAMS; i++)
      exam[i] = scanner.nextInt();
  }

  public void computeAverage() {
    double sum;
    sum = 0.0;
    for(int i=0; i<NUM_EXAMS; i++)
      sum = sum + exam[i];
    System.out.printf("Average score: %.2f \n",
    sum / NUM_EXAMS);
  }
}
```

1.7 Summary

- Big O notation is a valuable way of comparing the efficiency of different algorithms.
- The order of efficiency of algorithms from the fastest to the slowest is: O(1), O(log n), O(n), O(n log n), and $O(n^2)$.
- When there is more than one term in an expression after analysis of an expression, only the most dominate term (slowest) appears without coefficients in Big O notation.
- Data abstraction is the hiding of how the data is stored in a program.

- Procedural abstraction is the hiding of the algorithm.
- An abstract data type (ADT) is a class in which both the data and procedure are hidden or abstracted.
- An `interface` is a useful Java mechanism to implement ADTs.

1.8 Exercises (Items Marked with an * Have Solutions in Appendix B)

1. Evaluate the following code segments in terms of their algorithmic efficiency. Do not use Big O notation, but rather give a specific number for an answer as done in the first part of Sect. 1.3. (Note: Assume that i++ counts as 2 time units.)

```
*A. x = 0;
    for(i=0; i<3; i++)
       x = x + i * 2;
 B. a = 1;
    do {
       a = a + 2;
    } while(a <= 3);
 C. i = 1;
    y = 2;
    while(i <= 6) {
        i = i + 2;
        y = y * i;
    }
```

2. Evaluate the following code segments in terms of their algorithmic efficiency. Do not give a specific number, but rather use Big O notation in terms of n as done in the last part of Sect. 1.3.

```
*A.  i = 1;
     while(i <= n/2) {
         // some statements
         i = i + 1;
     }
 B.    j = 1;
```

```
        while(j <= n) {
           // some statements
           j = j * 2;
        }
C.    for(i=1; i<=n; i++)
          for(j=1; j<=n; j++)
             for(k=1; k<=n; k++) {
               // some statements
             }
```

3. Given the following expressions from analyzing algorithms, express them in Big O notation.

   (Hint: only the dominate term should appear within the parenthesis without any coefficients.)

   $*$A. $\log n + 2n + 3$ B. $n \log n + n^2 + 3n$ C. $\log n + n \log n + n$